

Cómo usa la IA las calculadoras (herramientas, agentes y filtro)

Guía de la pantalla **Funciones con IA** → **Asistente técnico**. Explica qué pasa desde que el usuario escribe hasta que aparece la respuesta, cómo se "presenta" cada calculadora a la IA, cómo se decide qué puede usar cada agente y qué hacer para agregar una herramienta nueva. Estado a 2026-09-24.

Si solo necesitas explicarle el flujo a alguien (sin entrar al código), hay un resumen de una hoja en `docs/como-funcionan-los-agentes-ia.md`. Este documento es la referencia completa.

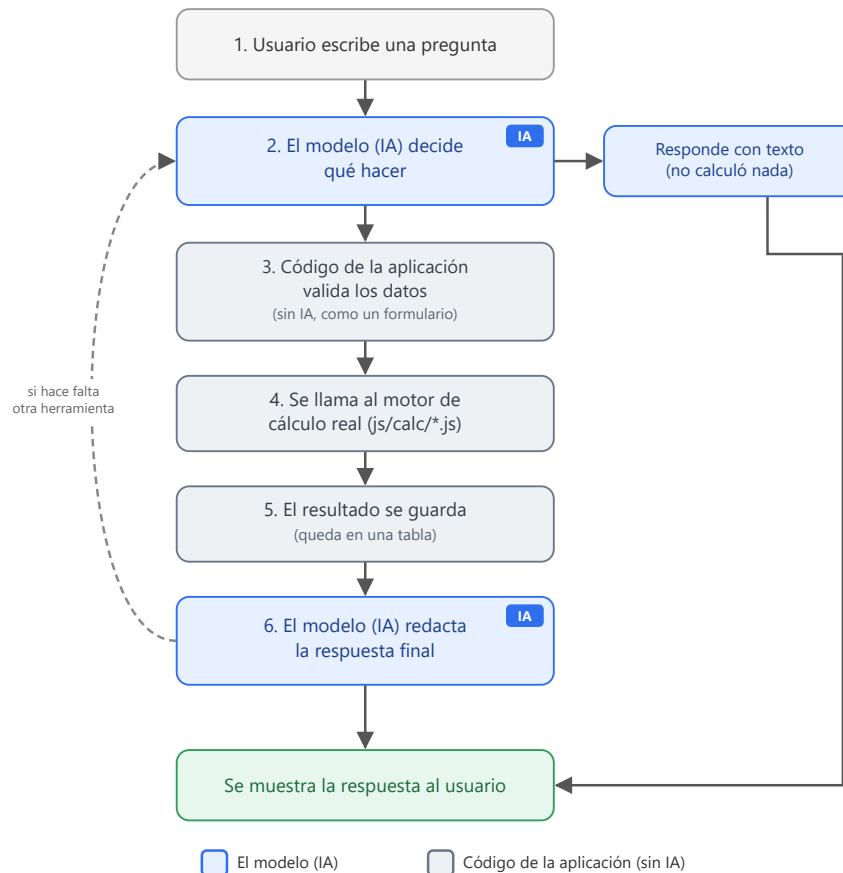
También hay una versión en PDF (`docs/ia-herramientas.pdf` , mismo contenido, cada sección numerada empieza en una página nueva) para compartir o imprimir sin pasar por el `.md` . Si se edita este archivo, regenerar el PDF con Edge headless (`--print-to-pdf`) — ver la sección "Verificación visual de cambios de UI" de `CLAUDE.md` .

1. La idea en una frase

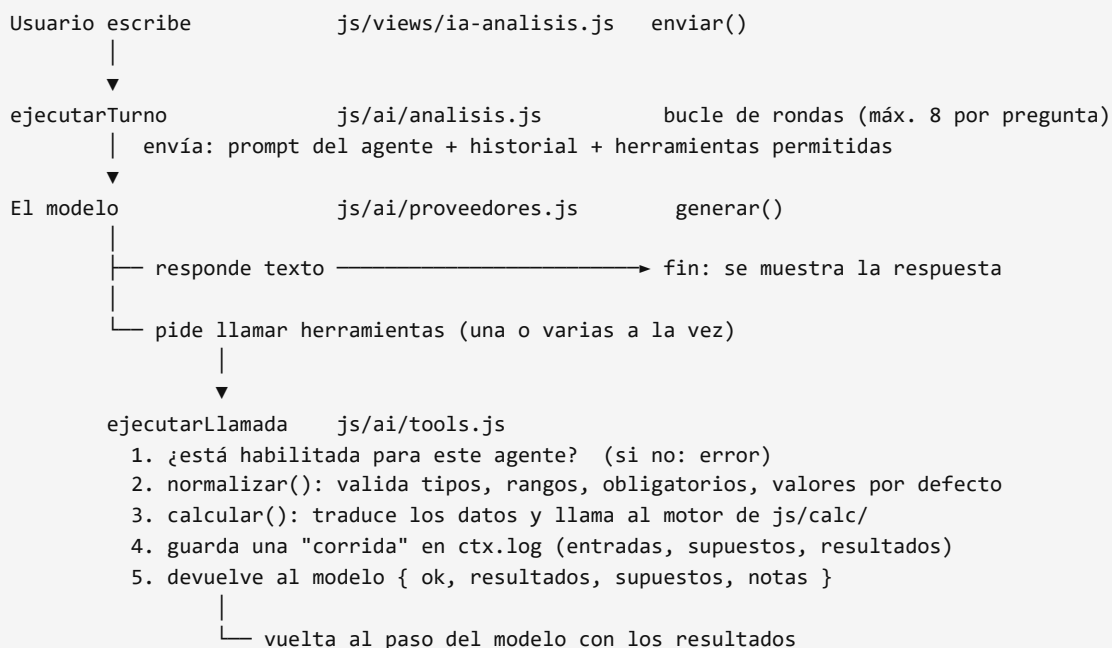
La IA nunca calcula. El modelo (Gemini, OpenAI o Claude, según el proveedor configurado en Configuración de IA) solo decide *qué herramienta usar y con qué datos*; los números salen de los mismos motores (`js/calc/*.js`) y catálogos (`data/*.json`) que usan las pantallas de Cálculos. La IA después interpreta esos resultados y redacta.

Esto se llama *function calling*: la app le describe al modelo un conjunto de funciones; el modelo responde "quiero llamar a `calcular_perdidas` con estos datos"; la app la ejecuta y le devuelve el resultado. Los tres proveedores usan este mismo mecanismo, cada uno con su propio formato de mensaje (`js/ai/gemini.js` , `openai.js` , `anthropic.js` , unificados por `js/ai/proveedores.js`).

2. Recorrido de una pregunta



Vista rápida arriba (mismo diagrama del resumen de una hoja); el detalle técnico —archivos, funciones y los 5 pasos internos de `ejecutarLlamada` — sigue abajo:



Detalles que importan:

- **Errores que la IA puede corregir.** Si los datos son inválidos, `ejecutarLlamada` **no lanza excepción**: devuelve `{ ok:false, error }` con un mensaje claro (qué falta, qué rango se permite, qué opciones existen) y el modelo reintenta con datos corregidos.
- **Límites por pregunta** (Funciones con IA → Configuración): `maxRondas` (idas y vueltas con el modelo, por defecto 8) y `maxCalculos` (cálculos individuales, por defecto 60). Un barrido de 10 puntos gasta 10. Si se agotan las rondas, se le pide un cierre sin más herramientas.
- **Progreso en pantalla.** `ejecutarTurno` emite eventos (`herramienta` al empezar, `herramienta-fin` con `ok` al terminar) y la vista dibuja las etiquetas `⊗ Regulación...` → `✓ Regulación` . Los títulos salen del campo `titulo` de cada herramienta.
- **Las tablas no las escribe la IA.** La sección "Cálculos ejecutados" y el reporte se dibujan con las *corridas* de `ctx.log` (`js/ai/reporte.js`). La IA solo aporta la narrativa. Por eso la IA no repite las tablas completas (regla 7 del prompt).

3. La "ficha" de una herramienta (`js/ai/tools.js`)

No hay archivo de configuración aparte: cada herramienta es un objeto JavaScript en `tools.js` . Tiene cuatro partes:

Parte	Para qué sirve
<code>nombre</code> , <code>titulo</code> , <code>descripcion</code>	Lo que el modelo lee para decidir si la usa (<code>descripcion</code>) y la etiqueta que ve el usuario (<code>titulo</code>).
<code>campos</code>	Los parámetros que el modelo puede enviar. De aquí se generan el esquema que recibe el modelo (<code>esquemaDe</code>) y la validación de lo que devuelve (<code>normalizar</code>).
<code>calcular(v, extra)</code>	El puente al motor: traduce los nombres de la IA a los del motor, busca en catálogos, aplica valores por defecto y llama a <code>js/calc/</code> .
Resultados	Lista de <code>res(clave, etiqueta, valor, unidad, decimales)</code> que devuelve <code>calcular</code> . Sus etiquetas salen en las tablas y las lee la IA.

Ejemplo (resumido; la real es `T_OCUPACION`):

```
const T_OCUPACION = {
  nombre: "calcular_ocupacion_ductos",
  tipo: "calculo",
  titulo: "Ocupación de ductos",
  descripcion: "Calcula el % de ocupación de un ducto ... El diámetro interno sale del catálogo o se ingresa con diametro_tubo_mm.",
  campos: [
    I("numero_conductores", "Número de conductores dentro del ducto", { e: "Número de conductores", req: true, min: 1, max: 9 }},
    N("diametro_conductor_mm", "Diámetro exterior de cada conductor", { u: "mm", req: true, min: 0, minExcl: true }},
    S("tipo_tuberia", "Tipo de tubería del catálogo ...", { oculto: true }},
  ],
  async calcular(v, extra) {
    // ... buscar el ducto en el catálogo, llamar a calcularOcupacionDuctos(...) ...
    return [res("ocupacion_pct", "Ocupación", r.porcentaje, "%"), /* ... */];
  },
};
```

Campos. Se crean con **N** (número), **I** (entero) y **S** (texto). Opciones: **req** (obligatorio), **min** / **max** / **minExcl**, **defecto**, **enum** (lista cerrada de valores), **u** (unidad), **e** (etiqueta corta para tablas; si falta se usa la descripción) y **oculto** (no se muestra como entrada en las tablas; se usa cuando **calcular** registra la entrada por su cuenta). Un valor con coma decimal ("0,5") se acepta; un parámetro desconocido se ignora y queda anotado; un **enum** se compara sin distinguir mayúsculas ni tildes.

Qué registra **calcular.** **extra.entradas** (datos usados que no venían tal cual del usuario, p. ej. la resistencia tomada del catálogo), **extra.supuestos** (valores por defecto o decisiones) y **extra.notas** (advertencias, p. ej. "los valores parecen grados"). Todo eso queda en la corrida y se ve en pantalla y en el reporte.

Tipos de herramienta (**tipo):**

- **calculo**: ejecuta un motor y registra una corrida (gasta 1 del presupuesto de cálculos).
- **consulta**: lee un catálogo y devuelve filas (no calcula ni gasta presupuesto, y no deja tabla en el reporte). **buscar_conductor**, **buscar_tuberia**.
- **barrido**: ejecuta una calculadora varias veces variando UN parámetro (hasta 40 puntos). Solo ofrece las calculadoras de **CALCULADORAS** (las 6 de Cálculos).
- **diseño**: "meta-herramienta" que combina calculadoras (ver más abajo); recibe **ctx** completo y arma su propio registro en el reporte.
- **ficha** (2026-09-23): registra/actualiza datos sin calcular (como **consulta**), pero Sí debe verse en el reporte (a diferencia de **consulta**, que se descarta a propósito): **guardar_ficha_proyecto**, la única de este tipo. No gasta presupuesto; deja un marcador mínimo en **ctx.log** ({ **ficha:true** }, sin entradas/resultados) que **armarTablas** ignora igual que a las consultas, porque su información vive aparte en **ctx.ficha** (ver más abajo).

Marcas opcionales de la ficha: **opcional: true** (no forma parte del agente estándar), **grupo** (dónde aparece la casilla en Agentes; por defecto se deduce del tipo) y, en los resultados, **cifras** (cuántas cifras significativas recibe la IA).

Precisión hacia la IA. Los números que recibe el modelo se redondean a **6 cifras significativas** (`redondear`). Para coordenadas y conversiones se usan 12 (`cifras`), porque con 6 un Este de 4 881 143 m quedaría con metros de error. La tabla que ve el usuario usa el valor completo.

Campos con lista de objetos (2026-09-19). Un campo puede ser una lista de objetos declarando `itemCampos` :

- `esquemaDe` convierte esos campos en un esquema anidado para el modelo, y `normalizar` valida cada elemento (errores del tipo `tramos[2]: "longitud_km" debe ser...`).
- Lo usan `tramos` (pérdidas y regulación), `grupos` (ocupación) y `puntos` (coordenadas).
- Regla común: lo que un tramo no indica se toma del nivel superior (así una línea de 3 tramos con el mismo conductor solo repite las longitudes).
- Helpers en `tools.js` : `campoTramos` , `listaTramos` , `volcarTramo` (antepone «Tramo N —» a entradas y notas), `datoPartida` .
- Los campos de nivel superior se conservan para el caso de un solo tramo/tipo/punto, de modo que las llamadas antiguas y `barrer_parametro` siguen funcionando.

4. Herramientas actuales (17)

Herramienta	Tipo	Grupo en Agentes	Agente estándar	Agente riguroso
<code>calcular_perdidas</code> , <code>calcular_regulacion</code> , <code>calcular_cortocircuito</code> , <code>calcular_ampacidad_aerea</code> , <code>calcular_ampacidad_subterranea</code> , <code>calcular_ocupacion_ductos</code> , <code>calcular_conductor_economico</code> , <code>valorar_alternativas</code>	calculo	Calculadoras	Sí	Sí
<code>buscar_conductor</code> , <code>buscar_tuberia</code>	consulta	Catálogos	Sí	Sí
<code>barrer_parametro</code>	barrido	Análisis	Sí	Sí
<code>dimensionar_conductor</code> , <code>verificar_conductor</code> , <code>resolver_valor_limite</code>	diseño	Análisis	No (opcionales)	Sí
<code>convertir_unidades</code> , <code>convertir_coordenadas</code>	calculo	Varios	No (opcionales)	Sí
<code>guardar_ficha_proyecto</code>	ficha	Análisis	No (opcional)	Sí

Dos agentes predefinidos (2026-09-23). `js/ai/agentes-analisis.js` ya no tiene un único predeterminado sino `AGENTES_PREDETERMINADOS` (array, ambos de solo lectura, viven en el código):

- **Agente estándar** (de siempre): usa `HERRAMIENTAS_ESTANDAR` , calcula rápido y declara los valores por defecto como *supuesto*.
- **Agente riguroso** (nuevo): usa `HERRAMIENTAS_TODAS` (las 17, incluidas las opcionales). Pensado para una *memoria de cálculo completa y definitiva* en vez de una estimación:
 - Antes de calcular pide todos los parámetros por categoría (Sistema, Conductor, Instalación...).
 - Avisa explícitamente cada valor por defecto y pide confirmarlo o cambiarlo (nunca lo asume en silencio).

- Si asume algo con su propio criterio (sin dato del usuario ni valor por defecto claro), lo marca como origen `estimado` y exige una `justificacion` (norma, rango típico, caso similar).
- Reparte las preguntas en varias respuestas para no saturar al usuario.
- Registra cada categoría confirmada con `guardar_ficha_proyecto`.
- Antes de calcular, revisa si algún dato luce atípico (aunque sea válido) y lo confirma con el usuario.
- Justo antes del cálculo definitivo, resume TODOS los parámetros y pide la confirmación final.
- Su reporte (`PROMPT_REPORTE_RIGUROSO` , en `js/ai/analisis.js`) pide una estructura de memoria de cálculo formal (normativa citada, metodología, verificación de cumplimiento...), distinta de la del estándar (`PROMPT_REPORTE`).
- `ID_PREDETERMINADO` / `AGENTE_PREDETERMINADO` siguen existiendo como alias del estándar (compatibilidad).

`guardar_ficha_proyecto` (2026-09-23; idea que estaba anotada como aplazada, "ficha del caso"):

- Registra por categoría los parámetros que el usuario confirmó, aceptó dejar en su valor por defecto, o que quedaron como una estimación de ingeniería justificada — campo `origen` : `usuario` | `defecto` | `estimado` (este último exige `justificacion` , agregado 2026-09-24).
- Campo `parametros` con `itemCampos` (`clave` , `etiqueta` , `valor` , `unidad` , `origen` , `justificacion`), mismo patrón que `tramos` / `grupos` / `puntos` .
- Guarda en `ctx.ficha` (un arreglo por categoría, mutado en sitio — igual que `ctx.log` , para que `conv.ficha = ctx.ficha` conserve la referencia entre turnos y al reabrir del historial).
- `js/ai/reporte.js` agrega `fichaHtml` / `fichaMd` , que arman la sección «Datos del proyecto» (una tabla por categoría, con el origen "Estimación (justificación)" cuando aplica) ANTES de «Cálculos ejecutados» en el reporte (Markdown, HTML exportable, impresión y `.docx`).
- Es opcional: no está en el agente estándar, solo la usa el riguroso (o un agente propio que la active).

`calcular_conductor_economico` (2026-09-22; entró al agente estándar el 2026-09-23, pedido del usuario):

- Compara entre 2 y 5 opciones de conductor de una línea nueva por su costo total actualizado (inversión + valor presente del costo de las pérdidas durante `anios`), igual que la calculadora.
- Reutiliza `js/calc/conductor-economico.js` (`compararOpciones` , `sensibilidad`) sin tocarlo; cada opción resuelve su conductor con el mismo `resolverConductor` / `resistencia75` que usan las demás fichas.
- NO entra en `CALCULADORAS` (sigue sin ofrecer barrido: su resultado es una comparación entre opciones, no un valor único que tenga sentido barrer).
- Devuelve por opción `opcionN_conductor/inversion/perdidas_pct/perdidas_mwh/costo_perdidas_vp/costo_total/compensa` , más `opcion_menor_costo` y `sensibilidad_robusta` (si la ganadora cambia en algún escenario de energía ± 10 % , demanda ± 10 % o tasa ± 2 puntos).
- Cuando el costo de instalación no se indicó en alguna opción (o en la ganadora), también agrega `opcionN_umbral_instalacion_km` por cada opción no ganadora afectada (`sensibilidadInstalacion` en `conductor-economico.js` , 2026-09-23): la DIFERENCIA de costo de instalación (\$/km) que haría cambiar la conclusión — no una estimación ni una dirección (no dice cuál instalación sería más cara, porque no se sabe); una nota se lo aclara al modelo para que no invente una dirección.

valorar_alternativas (2026-09-26, versión 3.30.0; en los DOS agentes, no es opcional):

- Es la pantalla «Valoración integral» para la IA: evalúa de 1 a 6 alternativas completas de una misma conexión con todos los criterios a la vez (ampacidad aérea IEEE 738 o subterránea IEC 60287, pérdidas, regulación, cortocircuito y costos) en UNA sola llamada, con el mismo motor (`js/calc/valoracion-integral.js` : `compararEscenarios` , `datosConductor` , `calibreMinimo` , `analizarAlternativa`), sin tocarlo.
- Obligatorio solo lo esencial: `factor_potencia` , `potencia_mw` o `potencia_mva` (sin corriente: depende de la tensión de cada alternativa), la tensión (`tension_kv` común o de cada alternativa) y, por alternativa, `red` + `material` + `calibre` y la longitud (común, de la alternativa o del tramo). Todo lo demás toma los valores por defecto de la pantalla (`DEF_VI`); solo se anotan como supuestos los que se usaron de verdad (los de aérea si hay tramos aéreos, los del terreno si hay subterráneos, los económicos si hay `precio_kwh`).
- Cada alternativa puede tener hasta 4 `tramos` en serie; lo que un tramo no indique lo hereda de su alternativa. Subterránea: cable monopolar en trébol del catálogo de construcción (nivel 15/35/46 kV según la tensión, `tipo_pantalla` , `aislamiento_pct`), varios conductores por fase = ternas en paralelo en el mismo banco (máximo 6 circuitos con `otros_circuitos`).
- Con `precio_kwh` y `costo_conductor_km` en todos los tramos de una alternativa, calcula su inversión y su costo total actualizado.
- Devuelve por alternativa: veredicto, corriente, ampacidad total y % de uso, pérdidas (% con su clase y kW), caída de tensión, capacidad de cortocircuito (y si soporta la falla), costos, el criterio que limita, la potencia máxima y la longitud máxima antes de incumplir, y el calibre mínimo del mismo tipo que cumple. Con 2 o más, `recomendada` (igual que la pantalla: entre las que cumplen, la de menor costo total, o la de menores pérdidas sin costos).
- NO entra en `CALCULADORAS` (sin barrido: su resultado es una comparación). Cuenta como un cálculo del presupuesto por pregunta.
- Los prompts la mencionan: el estándar (regla 2) para comparar alternativas completas o validar una de forma integral; el riguroso (regla 9) con los datos confirmados en la ficha.

Las de Varios no entran en el barrido de parámetros.

Herramientas de diseño (2026-09-20). Son «meta-herramientas»: no traen fórmulas, combinan las calculadoras (como el barrido). Son `opcional` : el agente estándar NO las usa; solo un agente propio las activa con su etiqueta, y necesita además habilitadas las calculadoras que ellas usan (si falta una, la herramienta lo dice). Cada llamada cuenta como UN cálculo del presupuesto por pregunta.

- `dimensionar_conductor` : recorre el catálogo de una familia/material (una fila por calibre, de menor a mayor área) y devuelve el MÁS PEQUEÑO que cumple todos los criterios, más la tabla de calibres evaluados y qué incumple cada uno. Los criterios se activan según los datos que lleguen: pérdidas (línea + longitud + `factor_carga`), regulación (más `dab_m/dac_m/dbc_m`), ampacidad (solo red Aérea, contra la corriente de carga) y cortocircuito (`corriente_falla_ka`). Límites por defecto = las referencias «Aceptable» (3 % pérdidas, 10 % regulación); una sola línea de un tramo.
- `verificar_conductor` : lo mismo para UN calibre (y `referencia` opcional); devuelve valor, límite, margen y veredicto por criterio.

- **resolver_valor_limite** : solucionador inverso por bisección: el valor de un parámetro numérico de una calculadora que deja un resultado justo en **valor_objetivo** dentro de **[minimo, maximo]** (p. ej. longitud máxima para 5 % de caída). Asume monotonía; si el objetivo no queda entre los extremos, lo avisa con los valores en ambos.
- En el reporte: **dimensionar_conductor** deja una fila por calibre (tabla «Dimensionar conductor» con «Cumple todos los criterios» Sí/No), **verificar_conductor** una fila, y el solucionador deja la fila de la calculadora en la solución.

Qué acepta y qué devuelve cada una (además de lo básico). Todas quedaron alineadas con las pantallas rediseñadas; los motores de **js/calc/** no cambiaron, se reutilizan los módulos de lógica de las pantallas:

Herramienta	Entradas nuevas	Resultados nuevos
calcular_perdidas	dato de partida potencia_mw / potencia_mva / corriente_a (exactamente uno), conductores_por_fase , tramos	perdidas_mw , potencia_activa_mw (si el dato no es MW), tramoN_* , clasificacion (Óptimo / Aceptable / Elevado) y nota con las referencias 1 % / 3 %
calcular_regulacion	lo mismo, más separacion_haz_m , rmg_m y distancias por tramo	tramoN_* , resistencia_efectiva_ohm_km y rmg_efectivo_mm (haz), clasificacion (referencias 5 % / 10 %)
calcular_cortocircuito	corriente_falla_ka (opcional)	cumple_corriente , margen_ka , area_minima_mm2 , calibre_sugerido (+ área y capacidad)
calcular_ampacidad_subterranea	—	monopolar: reactancia_mutua_ohm_m , resistencia_pantalla_ohm_m y corriente_circulante_pantalla_a (Ambos Extremos) o tension_inducida_pantalla_v_km (Unipuntual / Cross-bonding)
calcular_ocupacion_ductos	grupos (varios tipos; diámetro manual o catálogo XLPE con material + calibre y filtros opcionales)	radio_curvatura_mm (12D), total_conductores , area_total_mm2 , grupoN_*
convertir_unidades	catálogo data/unidades.json (19 categorías, cualquier unidad a cualquier otra; se reconoce símbolo, código o nombre); categoría Calibre (AWG/kcmil ↔ mm²) con el parámetro calibre	—
convertir_coordenadas	cualquiera de los ~509 códigos EPSG (ya no es un enum de 7), puntos (hasta 50)	puntoN_* ; notas de área de uso y de datum

Referencias de diseño. Óptimo / Aceptable / Elevado NO son límites normativos (el prompt estándar lo dice y la nota de cada resultado lo repite): nunca «fuera de norma».

5. Agentes y filtro de herramientas (**js/ai/agentes-analisis.js**)

Un **agente** es: nombre, descripción, prompt de sistema (`instrucciones`), instrucciones del reporte, temperatura opcional y **lista de herramientas que puede usar**. Se administra con el botón **Agentes** de la pantalla.

- **Agente estándar:** sale del código (`SISTEMA_ANALISIS` y `PROMPT_REPORTE` en `analisis.js`), es de solo lectura y nunca se escribe en `localStorage` . Si cambia el prompt estándar, se actualiza solo. Se puede ver y duplicar.
- **Agentes propios:** se guardan en `localStorage` (`ia.agentesAnalisis` ; el activo en `ia.agenteAnalisisActivo`). A su prompt la app siempre le agrega al final `REGLA_FIJA` (no calcular; todo número sale de una herramienta).
- **Agente guardado sin lista de herramientas** (de antes de existir el campo): usa las del estándar.

El filtro es de código, no solo de prompt. Hay tres capas:

1. `declaraciones(permitidas)` (`tools.js`): al modelo solo se le describen las herramientas del agente. El barrido solo ofrece las calculadoras permitidas y desaparece si no hay ninguna.
2. `ejecutarLlamada` con `ctx.permitidas` : si el modelo pide una herramienta no habilitada, se rechaza con "no está habilitada para este agente" y no se ejecuta nada.
3. Imports: `tools.js` importa únicamente los motores que expone. Lo que no está ahí no es alcanzable.

Refuerzo por prompt (regla 10 del estándar): si piden algo para lo que no hay herramienta, la IA no debe escribir ningún valor y debe remitir a Agentes. Es una instrucción al modelo: reduce el riesgo pero no lo elimina. Las tres capas anteriores garantizan que una herramienta no habilitada **nunca se ejecuta**, pero no impiden que el modelo escriba en su texto un número inventado; por eso existe esta regla y por eso conviene revisar la etiqueta ☒ de la herramienta y la sección "Cálculos ejecutados" cuando importe que un número provenga de un cálculo.

Si las instrucciones de un agente nombran una herramienta que desmarcó, la IA dirá que no la tiene (el formulario lo avisa).

6. Cómo agregar una herramienta nueva

No es automático, y es a propósito: hay que decidir con cuidado qué datos recibe, con qué rangos y cómo se interpreta cada resultado. Si no se registra, la calculadora sigue funcionando en la app; la IA simplemente no la conoce.

1. **Motor:** tiene que existir como módulo puro en `js/calc/` (sin DOM), importable desde `tools.js` .
2. **Ficha:** escribirla en `tools.js` (sección de calculadoras, o de "varios"): `nombre` (`calcular_...` o `convertir_...`), `titulo` , `descripcion` (qué hace, qué datos pide y qué devuelve, pensando en que la lee un modelo), `campos` y `calcular` . Replicar lo que hace la vista antes de llamar al motor (catálogos, valores por defecto). Lo que no venga del usuario, registrarlo como *supuesto*.
3. **Rangos y errores:** lanzar `ErrorHerramienta` con mensajes que digan qué corregir y qué opciones existen.
4. **Dónde registrarla:** en la lista `CALCULADORAS` (entra al barrido) o en `VARIOS` /registro propio (no entra al barrido). Añadirla a `REGISTRO` .
5. **Estándar u opcional:** por defecto márcala `opcional: true` (y `grupo` si no es de cálculo/catálogo). Solo se quita esa marca si se decide que el agente estándar la use. En ese caso revisar también el prompt

estándar.

6. **Cifras:** si sus resultados tienen magnitudes grandes con precisión importante (coordenadas, etc.), usar `cifras`.
7. **Pruebas:** agregar casos en `tools/verify_ia.html` contra el motor o una fuente independiente. Actualizar los conteos de herramientas que ya están en las pruebas.
8. **Publicación:** subir `CACHE_VERSION` en `sw.js` y actualizar la documentación (este archivo, README y CLAUDE.md).

Si se cambia la **firma de un motor** existente, hay que actualizar su ficha en `tools.js` y correr `verify_ia.html`.

7. Pruebas

`tools/verify_ia.html` es un arnés en el navegador (sin clave ni internet; Gemini simulado). Desde la raíz del proyecto:

```
python -m http.server 8000
# abrir http://localhost:8000/tools/verify_ia.html → lista PASS/FAIL
```

Secciones relacionadas con este tema: *tools: esquema para Gemini*, *tools: Varios*, *tools: herramientas permitidas por agente*, *análisis: bucle con herramientas y agentes de análisis*. `tools/verify_coordenadas.py` valida la conversión de coordenadas con una implementación independiente en Python.

8. Límites y cosas a recordar

- **Prompt ≠ garantía.** Las reglas del prompt orientan a la IA, pero no impiden que escriba números sin herramienta. Por eso el filtro y el reporte no dependen de lo que la IA diga: los valores de las tablas salen de las corridas.
- **Coordenadas por EPSG:** entre los 7 sistemas de siempre se usa el motor original; con cualquier otro código se usa proj4 (carga perezosa) y entre datums distintos la exactitud es del orden de metros (la herramienta lo avisa en las notas).
- **Longitud/latitud intercambiadas** dentro de rango (p. ej. 4.7 y -74.07) no se detectan; los nombres de los parámetros (`este_o_longitud`, `norte_o_latitud`) y la descripción reducen el riesgo.
- **Formato numérico en las respuestas de la IA:** no está normalizado; la IA puede escribir miles con coma o con punto. La app muestra los números con punto decimal y sin separador de miles.
- **No fijar nombres de modelo en el código:** se listan desde la API en Configuración.
- **Las fórmulas replican la app original** (Power Apps); algunas decisiones son intencionales (ver la sección "Decisiones de migración" del README).
- **La clave del proveedor de IA** es del propio usuario (guardada en su navegador). Solo Gemini admite además una clave del servidor (configurada por el administrador, vive solo en memoria); OpenAI y Claude son siempre BYOK local. Ver README, "Acceso con Google y Firebase".

9. Dónde está cada cosa

Archivo	Contenido
<code>js/ai/tools.js</code>	Fichas de las herramientas, esquema para el modelo, validación, ejecución, filtro por agente.
<code>js/ai/analisis.js</code>	Bucle de conversación con el modelo, prompt estándar (<code>SISTEMA_ANALISIS</code>) y del reporte (<code>PROMPT_REPORTE</code>), y los del riguroso (<code>SISTEMA_RIGUROSO</code> / <code>PROMPT_REPORTE_RIGUROSO</code>).
<code>js/ai/agentes-analisis.js</code>	Agentes: los 2 predefinidos, propios, herramientas por agente, <code>REGLA_FIJA</code> .
<code>js/ai/proveedores.js</code>	Registro de proveedores (Gemini/OpenAI/Claude): metadatos y cuál está activo.
<code>js/ai/gemini.js</code> , <code>openai.js</code> , <code>anthropic.js</code>	Cliente REST de cada proveedor, mismo contrato (<code>generar()</code> , <code>listarModelos()</code> ...).
<code>js/ai/reporte.js</code>	Tablas de "Cálculos ejecutados" y reporte a partir de las corridas.
<code>js/ai/historial.js</code>	Conversaciones guardadas (IndexedDB), incluidas sus corridas.
<code>js/views/ia-analisis.js</code>	Pantalla: chat, etiquetas de progreso, formulario de agentes.
<code>js/calc/*.js</code>	Motores de cálculo (los que ejecutan las herramientas).
<code>tools/verify_ia.html</code>	Pruebas del módulo de IA.